LIFEINTEL: A PRIVACY-PRESERVING UNIVERSAL PERSONAL INTELLIGENCE WORKSTATION WITH SEMANTIC DELTA EXTRACTION AND AUTONOMOUS CONSEQUENCE REASONING

Submitted by

NITHIN VIGNESH S

Reg. No (ASML24023)

of

AMET UNIVERSITY

A CAPSTONE PROJECT REPORT

Submitted to the

FACULTY OF ADVANCED COMPUTING SCIENCES

In partial fulfillment of requirements for the award of the degree

of

BACHELOR OF SCIENCE –

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

AMET UNIVERSITY

135, East Coast Road, Kanathur – 603 112

Tamil Nadu, India

NOVEMBER 2026



BONAFIDE CERTIFICATE

Certified that this project report titled **"LIFEINTEL: A Privacy-Preserving Universal Personal Intelligence Workstation with Semantic Delta Extraction and Autonomous Consequence Reasoning"** is the bonafide work of **NITHIN VIGNESH S (Reg No: ASML24023)** who carried out the project under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Supervisor
Mr. Tamilarasan D, M.CA,
Assistant Professor,
Department of ACS,
AMET University,
Kanathur - 603 112.

Head of the Department
Dr. C.N. Ravi, Ph.D.,
Head of the Department,
Department of CSE,
AMET University,
Kanathur - 603 112.

Submitted for Project Viva Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

In contemporary academic and professional workflows, critical obligations, deadlines and prerequisites are fragmented across unstructured documents such as institutional PDF circulars, Word specifications, emailed requirements and manual schedules. Conventional tools treat personal data as a passive document repository, a static calendar reminder or a cloud-based conversational assistant. These approaches do not, by themselves, recognise temporal conflicts between a revised document and a user's existing plans, they rarely highlight subtle revisions between document versions, and cloud-based assistants require sensitive documents to be sent to third-party services.

This capstone project presents LIFEINTEL (Universal Personal Intelligence System), a local-first, privacy-preserving desktop workstation. LIFEINTEL ingests local PDF, DOCX and TXT documents through an offline parsing pipeline. Regular-expression and rule-based entity extractors isolate dates, monetary amounts and obligation statements and normalise deadlines to ISO-8601. A Contextual Memory Engine builds an associative knowledge graph using term-frequency (TF) cosine similarity, a lexical-overlap measure, to link related documents without manual tagging.

A Delta and Consequence Reasoner compares successive document versions. It computes the temporal shift $\Delta T = T(D_2) - T(D_1)$ and checks stored user tasks against the new deadline using a deterministic, rule-based comparison. If a task is scheduled after the revised deadline, the system reports the shortfall in days and suggests a mitigation date. The reasoning is rule-based and transparent; the user remains the decision-maker.

The interface follows a "Warm Minimal" design philosophy. On a small internal benchmark of 10 documents, deadline extraction obtained precision 1.00, recall 1.00 with a mean extraction-algorithm latency of 3.15 ms per document on a CPU. Because of the small benchmark size, these figures show correct behaviour on the test set and are not claimed as general accuracy.

Keywords: personal intelligence, local-first, document change detection, deadline extraction, term-frequency cosine similarity, privacy.

ACKNOWLEDGEMENT

I offer my gratitude to the Lord Almighty who graced me and helped me all along to complete the project successfully. I would like to acknowledge the people who helped me to develop this project and who made it possible to do so.

First and foremost, I sincerely thank “Everlasting” for enabling me to complete this work successfully.

I owe my sincere thanks to Dr. C.N. Ravi, Ph.D., Head of the Department, AMET University, for taking a keen interest in the project and for his valuable guidance, coordination and timely suggestions and motivation to complete this project.

I express my sincere thanks to Mr. Tamilarasan D, M.CA Assistant Professor, Project Supervisor, Department of ACS, for his valuable guidance, coordination and efforts to bring out the best in me, his availability at all times and for providing timely questions and suggestions.

I am also very thankful to Mr. Aakaash M, MCA., Assistant Professor and Faculty Advisor, Department of ACS. I greatly benefited from his invaluable guidance and coordination.

I also express my thanks to all the staff members of the Department of ACS, AMET University, for their constant support and encouragement, and to all the non-teaching staff members for their support.

Finally, I express my gratitude to my parents and friends for their constant support and encouragement.

NITHIN VIGNESH S | [ASML24023]

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	ABSTRACT	1
	ACKNOWLEDGEMENT	2
	TABLE OF CONTENTS	3
	LIST OF FIGURES	6
	LIST OF TABLES	7
	ABBREVIATIONS	8
1	**INTRODUCTION**	9
	1.1 BACKGROUND AND MOTIVATION	9
	1.2 PROBLEM STATEMENT	9
	1.3 KEY TERMS USED IN THIS REPORT	9
	1.4 PROJECT OBJECTIVES	10
	1.5 PROJECT SCOPE	10
	1.6 DEGREE OF AUTOMATION IN “AUTONOMOUS” CONSEQUENCE REASONING	11
2	**SYSTEM ANALYSIS**	12
	2.1 EXISTING SYSTEM	12
	2.2 PROPOSED SYSTEM	13
	2.3 FEASIBILITY ANALYSIS	13
	2.4 LITERATURE SURVEY	14
3	**SYSTEM REQUIREMENTS**	15
	3.1 HARDWARE REQUIREMENTS	15
	3.2 SOFTWARE REQUIREMENTS	15
	3.3 FUNCTIONAL REQUIREMENTS	16
	3.4 NON-FUNCTIONAL REQUIREMENTS	16
	3.5 TECHNOLOGY STACK AND JUSTIFICATION	17
4	**SYSTEM DESIGN**	18
	4.1 ARCHITECTURE DIAGRAM	18
	4.2 UML DIAGRAMS	19
	4.2.1 USE CASE DIAGRAM	19
	4.2.2 CLASS DIAGRAM	20
	4.2.3 SEQUENCE DIAGRAM	20

CHAPTER	TITLE	PAGE NO
	4.3 MATHEMATICAL FORMULATIONS	21
	4.4 LEXICAL VERSUS SEMANTIC SIMILARITY	22
	4.5 ALGORITHM DESCRIPTIONS	22
	4.6 DATABASE DESIGN	23
	4.7 SECURITY AND PRIVACY DESIGN	24
	4.8 USER INTERFACE DESIGN	24
5	**MODULE DESCRIPTION**	26
	5.1 UNIVERSAL INGESTION AND DOCUMENT PARSER	26
	5.2 STRUCTURED INFORMATION EXTRACTION ENGINE	26
	5.3 SEMANTIC CHANGE DETECTOR AND CONSEQUENCE REASONER	27
	5.4 CONTEXTUAL MEMORY AND ASSOCIATIVE KNOWLEDGE GRAPH	27
	5.5 PROACTIVE FOLDER WATCHDOG SERVICE	27
	5.6 TRANSPARENCY AND DATA SOVEREIGNTY AUDIT CENTER	28
	5.7 NATURAL LANGUAGE WORKSTATION QUERY ENGINE	28
6	**SYSTEM IMPLEMENTATION**	29
	6.1 DEVELOPMENT ENVIRONMENT AND SETUP	29
	6.2 FRONTEND (REACT AND TYPESCRIPT)	29
	6.3 ELECTRON MAIN PROCESS AND PRELOAD	29
	6.4 FASTAPI BACKEND AND API ENDPOINTS	30
	6.5 SQLITE DATABASE INTEGRATION	30
	6.6 DOCUMENT INGESTION	31
	6.7 INFORMATION EXTRACTION	31
	6.8 CHANGE DETECTION AND CONSEQUENCE REASONING	31
	6.9 CONTEXTUAL MEMORY	31
	6.10 FOLDER MONITORING	32
	6.11 NATURAL LANGUAGE QUERY	32
	6.12 DATA EXPORT AND DELETION	32
7	**SYSTEM TESTING**	33
	7.1 UNIT TESTING	33

CHAPTER	TITLE	PAGE NO
	7.2 INTEGRATION TESTING	33
	7.3 ROBUSTNESS AND NEGATIVE TESTING	34
	7.4 PERFORMANCE BENCHMARKS	34
	7.5 END-TO-END CASE STUDY	35
	7.6 RESULTS AND DISCUSSION	36
8	**APPENDICES**	38
	8.1 SCREENSHOTS	38
	8.2 DATA DICTIONARY	40
9	**CONCLUSION AND FUTURE ENHANCEMENT**	43
	9.1 CONCLUSION	43
	9.2 LIMITATIONS	43
	9.3 FUTURE ENHANCEMENT	43
	REFERENCES	44

LIST OF FIGURES

FIGURE NO	NAME	PAGE NO
4.1	LIFEINTEL SYSTEM ARCHITECTURE	18
4.2	END-TO-END DATA FLOW (STEPS)	18
4.3	USE CASE DIAGRAM	19
4.4	CLASS DIAGRAM	20
4.5	SEQUENCE DIAGRAM	21
7.1	PYTEST OUTPUT	35
7.2	CASE STUDY RESULT	36
8.1	DASHBOARD	38
8.2	SOURCES AND FOLDER MONITORING	38
8.3	DOCUMENT LIBRARY AND UPLOAD	39
8.4	CONSEQUENCE BRIEFING	39
8.5	MEMORY EXPLORER	39
8.6	ASK LIFEINTEL	40
8.7	TRANSPARENCY CENTER	40

LIST OF TABLES

TABLE NO	NAME	PAGE NO
2.1	COMPARISON OF EXISTING APPROACHES AND LIFEINTEL	12
2.2	FEASIBILITY ANALYSIS	13
2.3	LITERATURE SURVEY	14
3.1	HARDWARE REQUIREMENTS	15
3.2	SOFTWARE REQUIREMENTS AND VERIFIED VERSIONS	15
3.3	FUNCTIONAL REQUIREMENTS	16
3.4	NON-FUNCTIONAL REQUIREMENTS	16
3.5	TECHNOLOGY STACK AND JUSTIFICATION	17
4.1	WORKED NUMERICAL EXAMPLES	22
4.2	SECURITY AND PRIVACY CONTROLS	24
4.3	USER INTERFACE SCREENS	24
5.1	UNIVERSAL INGESTION AND DOCUMENT PARSER – SPECIFICATION	26
5.2	STRUCTURED INFORMATION EXTRACTION ENGINE – SPECIFICATION	26
5.3	SEMANTIC CHANGE DETECTOR AND CONSEQUENCE REASONER – SPECIFICATION	27
5.4	CONTEXTUAL MEMORY AND ASSOCIATIVE KNOWLEDGE GRAPH – SPECIFICATION	27
5.5	PROACTIVE FOLDER WATCHDOG SERVICE – SPECIFICATION	27
5.6	TRANSPARENCY AND DATA SOVEREIGNTY AUDIT CENTER – SPECIFICATION	28
5.7	NATURAL LANGUAGE WORKSTATION QUERY ENGINE – SPECIFICATION	28
6.1	REST API ENDPOINTS	30
7.1	UNIT TEST CASES	33
7.2	INTEGRATION TEST CASES	33
7.3	ROBUSTNESS AND NEGATIVE TEST CASES	34
7.4	BENCHMARK DATASET DESCRIPTION	34
7.5	CASE STUDY TRACE	35
8.1	SOURCES	40
8.2	DOCUMENTS	41
8.3	EXTRACTED FACTS	41
8.4	OBLIGATIONS	41
8.5	DETECTED CHANGES	42

ABBREVIATIONS

ACID	-	ATOMICITY, CONSISTENCY, ISOLATION, DURABILITY
API	-	APPLICATION PROGRAMMING INTERFACE
CPU	-	CENTRAL PROCESSING UNIT
DOCX	-	OFFICE OPEN XML DOCUMENT FORMAT
FN / FP / TP	-	FALSE NEGATIVE / FALSE POSITIVE / TRUE POSITIVE
FR / NFR	-	FUNCTIONAL / NON-FUNCTIONAL REQUIREMENT
GGUF	-	GPT-GENERATED UNIFIED FORMAT
GPU	-	GRAPHICS PROCESSING UNIT
IPC	-	INTER-PROCESS COMMUNICATION
ISO	-	INTERNATIONAL ORGANIZATION FOR STANDARDIZATION
JSON	-	JAVASCRIPT OBJECT NOTATION
LLM	-	LARGE LANGUAGE MODEL
NLP	-	NATURAL LANGUAGE PROCESSING
OCR	-	OPTICAL CHARACTER RECOGNITION
ORM	-	OBJECT RELATIONAL MAPPING
PDF	-	PORTABLE DOCUMENT FORMAT
REST	-	REPRESENTATIONAL STATE TRANSFER
SHA	-	SECURE HASH ALGORITHM
SQL	-	STRUCTURED QUERY LANGUAGE
TF	-	TERM FREQUENCY
UI	-	USER INTERFACE
UML	-	UNIFIED MODELING LANGUAGE
UUID	-	UNIVERSALLY UNIQUE IDENTIFIER

CHAPTER 1
INTRODUCTION

1.1 BACKGROUND AND MOTIVATION

A personal intelligence system is software that observes a person's own information (documents, schedules, notes) and helps them act on it, rather than merely storing it. Early document-management tools only indexed files for search; calendar tools stored dates entered by hand; and recent cloud assistants can summarise a document on request. Students, researchers and professionals, however, receive a constant stream of information: administrative circulars, internship calls, syllabus guidelines and fee schedules. Missing a clause, or failing to notice that an updated circular moved a submission deadline earlier, can lead to academic penalties, financial loss or missed opportunities.

Existing productivity tools fall short in three ways:

- **Siloed awareness:** Calendars track dates, notes store text and file systems hold PDFs, but none of them cross-reference whether an update in one document invalidates a plan made elsewhere.
- **Privacy concerns:** Many AI assistants process documents on remote servers, so private transcripts, financial details and personal files leave the user's device. Some products offer on-device or enterprise-controlled modes, but the common consumer workflow is cloud-based.
- **Reactive rather than proactive behaviour:** Conversational assistants usually act when asked. They do not, by default, watch a local folder, notice that a new version of a file has arrived, and warn the user in advance.

1.2 PROBLEM STATEMENT

To design, implement and evaluate an offline personal intelligence system that extracts structured obligations from local documents, links related documents, calculates the effect of deadline revisions on the user's existing tasks, and alerts the user through a desktop workstation interface, without sending any data to external services.

1.3 KEY TERMS USED IN THIS REPORT

- **Local-first architecture:** All data, processing and storage stay on the user's own machine; the application works with the network disconnected.

- **Privacy-preserving intelligence:** Design in which personal documents are never transmitted to a third party. In LIFEINTEL this means zero outbound network calls from the processing pipeline, which can be checked in the Transparency Center and with a network monitor.
- **Semantic delta extraction:** The extraction of the difference (delta) between two versions of a document, such as a changed deadline, fee or requirement. In this project the comparison is performed on extracted structured facts using rules; it is not a neural semantic-understanding model.
- **Consequence reasoning:** Rule-based checking of what a detected change implies for the user's stored tasks, for example a task now scheduled after a revised deadline.

1.4 PROJECT OBJECTIVES

Each objective is stated with a measurable acceptance criterion.

No.	Objective	Measurable criterion
O1	Local-only processing	Zero outbound network connections during ingestion, extraction, reasoning and query (checked with the Transparency Center counter and an OS network monitor).
O2	Multi-format ingestion with deduplication	PDF, DOCX and TXT files parsed; a byte-identical file uploaded twice is indexed once (SHA-256).
O3	Structured fact extraction	Deadline extraction precision and recall reported on a labelled benchmark with the dataset size and ground-truth method stated.
O4	Consequence and conflict reasoning	Every deadline moved earlier than an existing task's date is reported with the correct shortfall in days on the test scenarios.
O5	Associative memory	Document pairs with TF cosine similarity $\geq \theta = 0.15$ appear as graph edges with their weights.
O6	Proactive monitoring	A file added to an approved folder is detected and queued after the 1.5 s debounce without user action.
O7	User control of data	The user can export all stored data as JSON and permanently purge all tables.
O8	Usable interface	Seven screens operate correctly at 1440 × 900 resolution.

1.5 PROJECT SCOPE

What LIFEINTEL does:

- Parses PDF, DOCX and TXT documents from user-approved folders or manual upload.

- Extracts dates, monetary amounts and obligation statements with confidence scores.
- Detects duplicate files and new versions of earlier documents and reports differences.
- Flags deadline conflicts with the user's stored obligations and suggests a mitigation date.
- Links related documents by lexical similarity and answers questions from stored facts with source citations.

What LIFEINTEL does not do:

- It does not understand meaning beyond the words present in the text (see Section 4.4); synonyms such as "due date" and "deadline" are not treated as equivalent by the similarity measure.
- It does not read text from scanned images in the current build.
- It does not email, edit or submit anything on the user's behalf; it only alerts and suggests.
- It is not a replacement for official university communication; extraction can be wrong, so the user should confirm important dates in the source document.

1.6 DEGREE OF AUTOMATION IN "AUTONOMOUS" CONSEQUENCE REASONING

The word "autonomous" in the title means that, once a document arrives, the pipeline from parsing to alert runs without user input. The decision logic is a deterministic rule (Section 4.3): a conflict exists when an existing task's date is later than the new deadline. The system never changes a task, deadline or file by itself. It produces a briefing and a suggested date, and the user decides whether to act. Full automation therefore covers detection and alert generation; the final decision remains human. Extraction and consequence conflict detection are executed strictly via regex pattern matching and deterministic heuristic rules, ensuring reliable execution without requiring continuous local LLM inference.

CHAPTER 2
SYSTEM ANALYSIS

2.1 EXISTING SYSTEM

- **Generic productivity dashboards:** Tools such as Notion, Trello and Google Keep depend on manual data entry. If a PDF notice contains ten requirements and an updated date, the user must read and type each item.
- **Cloud AI assistants:** ChatGPT, Claude and Copilot can summarise documents, typically after a manual upload, and mostly run on cloud servers (some offer enterprise or on-device options). They generally do not watch a local folder or keep a persistent model of the user's schedule conflicts unless configured to.
- **Calendar and task managers:** Google Calendar and Outlook record fixed points in time but do not read document text or reason about consequences.

COMPARISON OF APPROACHES

TABLE 2.1 COMPARISON OF EXISTING APPROACHES AND LIFEINTEL

Feature	Traditional tools	Cloud AI assistants	LIFEINTEL
Document parsing	Limited	Yes	Yes (PDF, DOCX, TXT)
Automatic deadline extraction	Limited	Possible on request	Yes (rule-based)
Local processing	Varies by tool	Usually cloud; some local or enterprise modes	Yes
Document revision detection	Version history only	Manual prompting	Yes (hash and fact comparison)
Cross-document relationships	Limited	Depends on context provided	Yes (lexical TF cosine)
Proactive conflict alerts	Limited (reminders)	Not inherent	Yes

LIMITATIONS OF EXISTING SYSTEMS

- Revisions buried in multi-page documents are routinely overlooked.
- Sensitive student and financial documents may be exposed to third-party services.
- There is little automatic correlation between external notices and personal milestones.

2.2 PROPOSED SYSTEM

LIFEINTEL is a cognitive layer that runs on the local operating system. It ingests files from approved folders through a watchdog monitor, extracts facts using local pattern-based NLP, detects revisions through SHA-256 hashing and title/fact comparison, and compares incoming constraints with stored user tasks to produce Consequence Intelligence Briefings, with optional local quantized LLaMA-3 execution supported for natural-language query summarization.

ADVANTAGES

- Data stays on the device; no network connection is required for processing.
- Dates, fees and prerequisite obligations are structured and saved automatically.
- Bottlenecks are flagged before they happen, for example: “Resume draft scheduled for Nov 12 conflicts with the revised deadline of Nov 10.”

LIMITATIONS

- Extraction depends on writing patterns the rules recognise; unusual date phrasing may be missed or misread.
- Lexical similarity does not capture meaning; related documents that use different words may not be linked.
- Performance and accuracy were measured on a small benchmark and have not been tested on a large, varied document collection.
- Current build targets Windows.

2.3 FEASIBILITY ANALYSIS

TABLE 2.2 FEASIBILITY ANALYSIS

Type	Assessment
Technical	All components (Electron, React, FastAPI, SQLite, PyMuPDF, python-docx, watchdog) are mature open-source tools that run on an ordinary laptop CPU without a GPU.
Operational	The user drops files into a folder or uploads them; alerts appear on the dashboard and as desktop notifications. No configuration of servers or accounts is needed.
Economic	All software used is free and open source and no cloud or API fees are incurred. The only cost is the user’s existing computer and about 2 GB of disk space.

2.4 LITERATURE SURVEY

LIFEINTEL builds on established ideas from information retrieval, temporal reasoning and language modelling. Table 2.3 summarises the works that influenced its design and how each is used or deliberately not used.

TABLE 2.3 LITERATURE SURVEY

Reference	Contribution	Relevance to LIFEINTEL
Salton, Wong and Yang (1975) [4]	Vector space model: documents as term vectors compared by cosine similarity.	Basis of the Contextual Memory Engine (TF vectors, cosine, threshold 0.15).
Allen (1983) [6]	Formal relations between time intervals (before, after, overlaps).	Motivates reasoning about deadlines and task dates; LIFEINTEL uses the simple point-in-time 'before / after' case.
Lewis et al. (2020) [5]	Retrieval-augmented generation: answers conditioned on retrieved documents.	Inspires answering questions only from retrieved database records with citations.
Devlin et al. (2019) [1]	BERT: contextual embeddings capturing word meaning.	Not used in the current build; proposed to replace TF vectors for true semantic similarity.
Touvron et al. (2023) [3]	LLaMA: open foundation language models that run locally when quantised.	Candidate local model for optional language tasks without cloud access.
Radford et al. (2023) [2]	Whisper: robust speech recognition.	Planned future module for lecture and voice-note obligations.

Gap identified: the surveyed works address similarity, time reasoning, retrieval and language modelling separately. None is packaged as a local, document-watching tool that connects a document revision to a user's own task schedule. LIFEINTEL combines these ideas in a small, transparent pipeline aimed at that gap.

CHAPTER 3

SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS

TABLE 3.1 HARDWARE REQUIREMENTS

Item	Minimum	Recommended
Processor	Intel Core i3 / AMD Ryzen 3, 4 cores, 64-bit	Intel Core i5 (8th Gen+) / AMD Ryzen 5, 6 cores (development machine: AMD Ryzen 5 PRO 8640HS)
RAM	8 GB (documents, database and UI only)	16 GB (needed if the local LLM is enabled)
Storage	2 GB free on SSD or HDD	512 GB NVMe SSD with 10 GB free (LLM model files add several GB)
GPU	Not required	Not required; integrated graphics (Radeon 760M / Intel Iris Xe) is sufficient
Network	Not required during use	Needed only to install dependencies and download the model once

3.2 SOFTWARE REQUIREMENTS

TABLE 3.2 SOFTWARE REQUIREMENTS AND VERIFIED VERSIONS

Component	Version in report	Version installed (from package.json / pip freeze)
Operating system	Windows 10 / 11 (64-bit)	Windows 11 (64-bit, Build 26200)
Electron	41.2.0	41.10.7
React	19.3.1	19.3.1
TypeScript	7.0	7.0.2
Vite	8.3.5	8.3.5
Tailwind CSS	3.4	3.4.19
Python	3.10 – 3.14	3.12.10
FastAPI / Starlette / Uvicorn	—	FastAPI 0.115.12 / Starlette 0.46.2 / Uvicorn 0.34.3
SQLite / SQLAlchemy	SQLite 3 / 2.0	SQLite 3.49.1 / SQLAlchemy 2.0.41
PyMuPDF / python-docx / Pillow	—	PyMuPDF 2.1 / python-docx 1.2.0 / Pillow 13.0.0
watchdog	—	7.0.0
Ollama and model	llama3:8b-instruct-q4_K_M	Ollama (Optional local LLaMA-3 8B Q4_K_M)
OCR engine	None named in earlier draft	None (Direct digital text extraction via PyMuPDF/python-docx)

Component	Version in report	Version installed (from package.json / pip freeze)
pytest / AnyIO	9.2	pytest 9.2.0 / AnyIO 4.16.0

3.3 FUNCTIONAL REQUIREMENTS

TABLE 3.3 FUNCTIONAL REQUIREMENTS

ID	Requirement
FR-01	The user can upload documents (PDF, DOCX, TXT) through the interface.
FR-02	The system extracts dates, monetary amounts and obligations from documents with confidence scores.
FR-03	The system detects duplicate documents using SHA-256 hashes and does not index them twice.
FR-04	The system identifies a new document as a revision of an earlier one and records the differences.
FR-05	The system detects deadline conflicts between a revised deadline and stored obligations.
FR-06	The system generates a consequence briefing stating the shift, the shortfall in days and a suggested date.
FR-07	The system monitors only folders explicitly approved by the user.
FR-08	The user can explore relationships between documents in a memory graph.
FR-09	The user can ask questions and receive answers with source citations from stored data.
FR-10	The user can export all stored data as JSON and permanently delete it.
FR-11	The user can add, edit and delete obligations manually.

3.4 NON-FUNCTIONAL REQUIREMENTS

TABLE 3.4 NON-FUNCTIONAL REQUIREMENTS

Category	Requirement
Privacy and security	No document content leaves the device; the API listens on localhost only; Electron uses context isolation and a preload bridge.
Performance	Extraction of a typical document completes in milliseconds on a CPU; UI remains responsive during background ingestion.
Reliability	The application recovers from unreadable files, folder-permission errors and database errors without crashing, and reports the failure to the user.
Usability	Seven clearly named screens; readable at 1440 × 900; alerts ordered by urgency.
Offline operation	All features work with the network disabled.
Category	**Requirement**

Data integrity	ACID transactions in SQLite; unique constraint on file hash; cascade deletion avoids orphan records.
Maintainability	Separate modules for parser, extractor, change detector, reasoner, memory and monitor; type-checked frontend; automated tests.

3.5 TECHNOLOGY STACK AND JUSTIFICATION

TABLE 3.5 TECHNOLOGY STACK AND JUSTIFICATION

Layer	Technology	Justification in LIFEINTEL
Desktop shell	Electron (CJS)	Native Windows integration: frameless window, system tray, desktop notifications and shell.openPath.
Frontend	React + TypeScript	Reusable components, compile-time type safety and reactive state.
Bundler	Vite	Fast hot module replacement and optimised production build.
Styling	Tailwind CSS	Implements the Warm Minimal tokens: Ivory #FBF9F5, Espresso #1A1613, Sage #5F8A5F, Terracotta #C97043.
Backend	FastAPI (Python)	Asynchronous REST endpoints, OpenAPI documentation and Pydantic validation.
Persistence	SQLite + SQLAlchemy	Serverless, zero-configuration, ACID-compliant local database.
Parsing	PyMuPDF, python-docx	Fast extraction of text and page offsets from PDF and Word files.
Watcher	watchdog	Event-driven monitoring with a debounce queue.
Memory	TF cosine similarity	Lexical similarity without GPU cost; interpretable weights.

Electron embeds Chromium and Node.js and hosts the interface; it communicates with the renderer through a context-isolated preload bridge. React and TypeScript build the dashboard, Memory Explorer, Transparency Center and query screen as typed components. FastAPI, built on Starlette and Pydantic, exposes the processing pipeline through local REST endpoints served by Uvicorn. SQLite with SQLAlchemy stores documents, facts, obligations, changes and logs. PyMuPDF extracts text from PDFs and python-docx reads Word files. Watchdog uses the Windows ReadDirectoryChangesW API with a 1.5 second debounce. Ollama / llama.cpp can run a 4-bit quantised LLaMA-3 model [3] on the CPU for local language tasks.

CHAPTER 4
SYSTEM DESIGN

4.1 ARCHITECTURE DIAGRAM

LIFEINTEL has four decoupled tiers: the Acquisition tier (approved-folder watchers and drag-and-drop upload), the Cognitive tier (parser, extractor, change detector, consequence reasoner, contextual memory), the Persistence tier (SQLite with SQLAlchemy) and the Presentation tier (Electron shell with the React interface, backed by the FastAPI service).

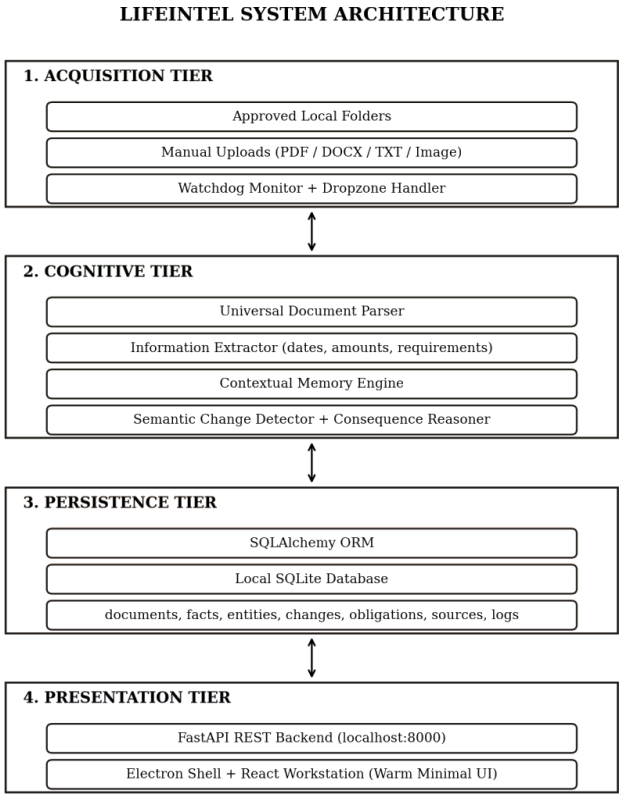


FIG 4.1 ARCHITECTURE DIAGRAM

COMPLETE DATA FLOW

- **Step 1.** The user uploads a document or drops it in an approved folder, and the watchdog queues it after the 1.5 s debounce.
- **Step 2.** The parser computes the SHA-256 hash (rejecting exact duplicates) and extracts text and page offsets.

- **Step 3.** The information extractor identifies dates, amounts and obligations and assigns confidence scores.
- **Step 4.** The contextual memory engine builds a TF vector and compares it with existing documents to create relationships.
- **Step 5.** The change detector looks for an earlier version with similar title tokens and compares the facts of both versions.
- **Step 6.** The consequence reasoner checks the new deadline against existing obligations and computes any shortfall.
- **Step 7.** Facts, changes, obligations and briefings are written to SQLite.
- **Step 8.** The React dashboard fetches the results through the REST API and shows alerts and a desktop notification.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

The user authorises folders, uploads documents, reviews facts and briefings, explores the memory graph, asks questions, manages obligations, and exports or purges data. The watchdog service triggers automatic scans.

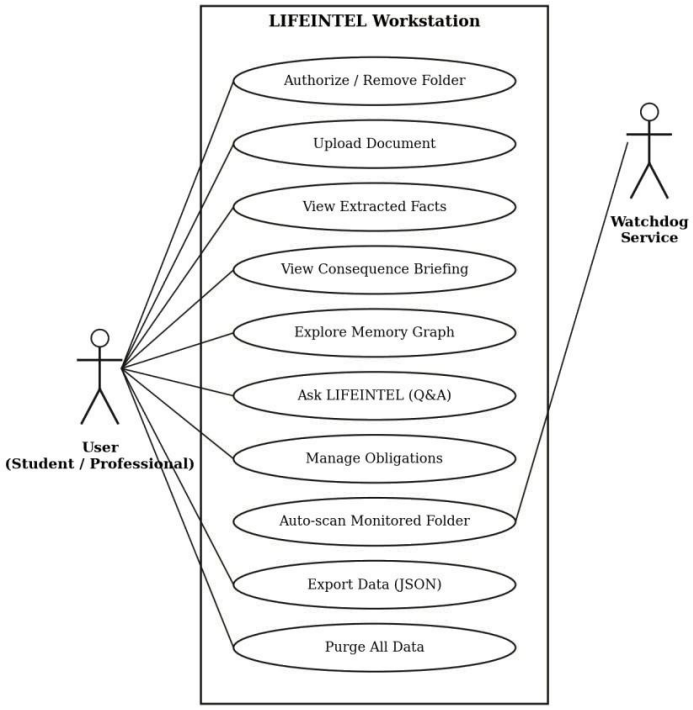


FIG 4.3 USE CASE DIAGRAM

4.2.2 CLASS DIAGRAM

A Source contains many Documents; each Document yields many Extracted Facts; successive versions produce Detected Changes; Obligations may originate from a Document; the Contextual Memory component indexes documents.

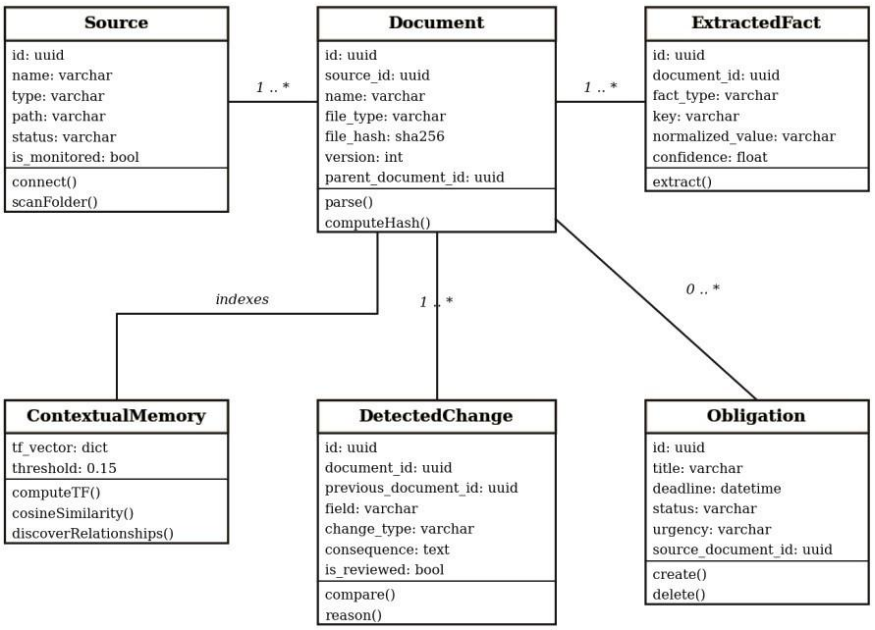


FIG 4.4 CLASS DIAGRAM

4.2.3 SEQUENCE DIAGRAM

The sequence diagram traces a document from the moment it is dropped in an approved folder to the moment the user is notified of a consequence.

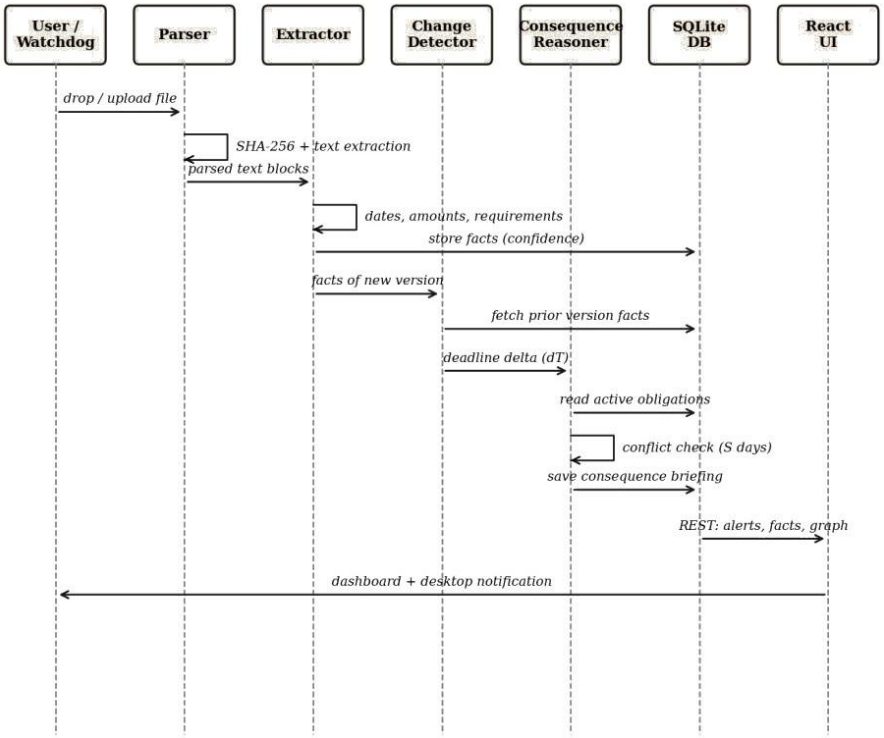


FIG 4.5 SEQUENCE DIAGRAM

4.3 MATHEMATICAL FORMULATIONS

1. Temporal delta and shift

When a document version D_2 supersedes D_1, with deadlines $T(D_2)$ and $T(D_1)$, the shift is $\Delta T = T(D_2) - T(D_1)$. If $\Delta T < 0$ the deadline moved earlier by $|\Delta T|$ days (urgency: critical). If $\Delta T > 0$ it was extended (urgency: low).

Example: old deadline $T(D_1)$ = 15 November, new deadline $T(D_2)$ = 10 November. $\Delta T = 10 - 15 = -5$, so the deadline moved five days earlier and urgency is critical.

2. Consequence conflict condition

For an existing task P with scheduled date T(P), a conflict exists if $T(P) > T(D_2)$. The shortfall is $S = T(P) - T(D_2)$ days, and the suggested mitigation date is $T(\text{mitigation}) = T(D_2) - 2$ days.

Example: the existing task "Resume draft" is planned for 12 November and the new deadline is 10 November. Since 12 > 10, there is a conflict; $S = 12 - 10 = 2$ days; the suggested mitigation date is $10 - 2 = 8$ November.

3. Term-frequency cosine similarity

Each document is represented as a vector of term counts over the normalised vocabulary. The similarity between documents A and B is Similarity(A, B) = (A · B) / (‖A‖ ‖B‖) [4]. A relationship is created when Similarity ≥ θ, with θ = 0.15.

Example: let the vocabulary be (internship, resume, deadline, fee). Document A = (2, 1, 1, 0) and Document B = (1, 0, 1, 1). A · B = 2×1 + 1×0 + 1×1 + 0×1 = 3. ‖A‖ = √(4+1+1) = √6 ≈ 2.449 and ‖B‖ = √(1+0+1+1) = √3 ≈ 1.732. Similarity = 3 / (2.449 × 1.732) = 3 / 4.243 ≈ 0.707. Since 0.707 ≥ 0.15, the two documents are linked with weight 0.707.

TABLE 4.1 WORKED NUMERICAL EXAMPLES

Formula	Inputs	Result	Meaning
ΔT	$T(D_1)$=15 Nov, $T(D_2)$=10 Nov	−5 days	Deadline moved 5 days earlier
S	T(P)=12 Nov, $T(D_2)$=10 Nov	2 days	Task is 2 days too late; suggest 8 Nov
Cosine	A=(2,1,1,0), B=(1,0,1,1)	≈ 0.707	Above θ = 0.15, so related

4.4 LEXICAL VERSUS SEMANTIC SIMILARITY

The Contextual Memory Engine uses term-frequency cosine similarity. This measures lexical overlap, that is, how many of the same words two documents share, weighted by how often they occur. It does not model meaning. Two documents that use the same words in different senses can score high, while two documents that express the same idea with different words (for example “deadline” and “last date to apply”) can score near zero. The edges in the Memory Explorer should therefore be read as “textually related” rather than “semantically equivalent”.

The word “semantic” in the project title refers to the extraction of meaningful facts (dates, amounts, obligations) and the comparison of those facts between versions, not to a learned semantic embedding. Sentence-embedding models such as BERT [1] could provide true semantic similarity and are proposed as future work (Section 9.2).

4.5 ALGORITHM DESCRIPTIONS

This section gives the step-by-step logic of the main processing stages in pseudocode.

Algorithm 1: Document ingestion pipeline

```
INPUT: file path p
1.  digest <- SHA256(bytes of p)
2.  IF digest exists in documents.file_hash THEN return DUPLICATE
3.  text, pages <- PARSE(p)              // PyMuPDF / python-docx / plain read
```

```
4.  IF text is empty THEN return ERROR(unreadable)
5.  facts <- EXTRACT(text, pages)         // dates, amounts, obligations
6.  doc <- STORE(document, facts)
7.  prev <- FIND_PREVIOUS_VERSION(doc)    // shared title tokens
8.  IF prev exists THEN
        changes <- COMPARE(prev.facts, doc.facts)
        FOR each deadline change: briefing <- REASON(change, active
obligations)
9.  UPDATE_MEMORY_GRAPH(doc)
10. return doc, changes, briefings
```

Algorithm 2: Deadline extraction

```
INPUT: text blocks with page numbers
FOR each block:
    FOR each match of ISO_DATE or text-date pattern:
        d <- NORMALISE(match)                  // YYYY-MM-DDTHH:MM:SS
        C <- 0.95 if explicit ISO date else lower value for free-text date
        key <- NEAREST_TRIGGER_WORD(match)     // e.g. application deadline
        SAVE fact(type=deadline, key, value, d, C, page)
    FOR each match of AMOUNT pattern: SAVE fact(type=amount, ...)
    FOR each trigger phrase (must submit, mandatory, prerequisite):
        SAVE fact(type=requirement, ...)
```

Algorithm 3: Term-frequency vector and cosine similarity

```
TF(text):  tokens <- lowercase words, remove stop words
           return map token -> count
COS(A, B): dot <- sum over shared tokens of A[t] * B[t]
           IF |A| = 0 OR |B| = 0 THEN return 0
           return dot / (sqrt(sum A[t]^2) * sqrt(sum B[t]^2))
```

The time complexity of building the relationship graph is O(nZ · v) for n documents and an average of v distinct terms per document. This is acceptable for a personal collection of a few hundred documents; a larger collection would need an inverted index or approximate nearest-neighbour search.

4.6 DATABASE DESIGN

The database has seven tables: sources, documents, extracted_facts, entities, detected_changes, obligations and activity_logs. The main relationships are as follows.

- One source has many documents (documents.source_id references sources.id).
- One document has many extracted facts (extracted_facts.document_id).
- A document may point to its earlier version (documents.parent_document_id), forming a version chain.
- A detected change references both the new document and the baseline document.
- An obligation may reference the document it was extracted from (obligations.source_document_id); manual obligations leave this empty.

Deleting a source cascades to its documents and facts, so no orphan records remain. The UNIQUE constraint on file_hash enforces deduplication at the database level even if the application check is bypassed. The field-level definitions are given in Section 8.2.

4.7 SECURITY AND PRIVACY DESIGN

TABLE 4.2 SECURITY AND PRIVACY CONTROLS

Threat	Control in LIFEINTEL
Document content sent to a third party	No outbound calls in the processing pipeline; the Transparency Center shows the external-call counter.
Other machines reaching the API	FastAPI is bound to localhost (127.0.0.1) only.
Malicious script in the interface reaching the system	Electron context isolation, node integration disabled, and a minimal preload bridge.
Reading unapproved folders	Watchdog observes only folders the user has authorised; the user can remove a folder at any time.
Loss of control over stored data	One-click JSON export and permanent purge of all tables.
Corrupted or malicious files	Unsupported types are rejected; parse errors are caught and reported without stopping the service.

Limitation: the SQLite file is not encrypted in the current build, so protection against someone with access to the user's operating-system account depends on the operating system. Database encryption is a possible enhancement.

4.8 USER INTERFACE DESIGN

The "Warm Minimal" design favours a clear information hierarchy and high density over decorative effects. Screens are designed for 1440 × 900 resolution, use a single typeface family, and use colour only to indicate meaning, for example urgency levels.

TABLE 4.3 USER INTERFACE SCREENS

Screen	Purpose	Main elements
Dashboard	Overview of what needs attention	Upcoming deadlines, critical alerts, recent documents
Sources	Manage approved folders	Add or remove folder, watchdog status, last sync
Document Library	Upload and review documents	Upload area, document list, extracted facts with confidence
Consequence Briefings	Understand the effect of a revision	Old and new value, shortfall in days, suggested date, mark as reviewed
Memory Explorer	See related documents	Node-and-edge graph with similarity weights

Screen	Purpose	Main elements
Ask LIFEINTEL	Question answering	Question box, answer, source citations
Transparency Center	Verify privacy and control data	Audit counters, database path, export, purge

CHAPTER 5

MODULE DESCRIPTION

LIFEINTEL has seven modules. Each is described below by its purpose, input, processing and output.

5.1 UNIVERSAL INGESTION AND DOCUMENT PARSER

Implemented in document_parser.py.

TABLE 5.1 UNIVERSAL INGESTION AND DOCUMENT PARSER – SPECIFICATION

Item	Description
Purpose	Convert any supported file into normalised text with metadata and a unique fingerprint.
Input	PDF, DOCX and TXT files, from upload or the watched folder.
Processing	Text extraction (PyMuPDF for PDF, python-docx for DOCX, direct read for TXT); SHA-256 hashing; page-offset recording; duplicate check against file_hash.
Output	Normalised text blocks with page numbers, file metadata and hash; a duplicate alert if the hash already exists.

Image files are not converted to text in the current build because no OCR engine is integrated.

5.2 STRUCTURED INFORMATION EXTRACTION ENGINE

Implemented in information_extractor.py.

TABLE 5.2 STRUCTURED INFORMATION EXTRACTION ENGINE – SPECIFICATION

Item	Description
Purpose	Turn text into structured facts: deadlines, amounts and obligations.
Input	Normalised text blocks from the parser.
Processing	Regular expressions with negative lookbehinds recognise ISO-8601 dates, currency amounts (₹, $, €, £, INR, USD) and trigger phrases such as “must submit”, “mandatory” and “prerequisite”. Dates are normalised to YYYY-MM-DDTHH:MM:SS. A confidence score C ($0.0 \leq C \leq 1.0$) reflects the certainty of the syntactic match.
Output	Rows in extracted_facts (type, key, value, normalised value, confidence, page number).

5.3 SEMANTIC CHANGE DETECTOR AND CONSEQUENCE REASONER

Implemented in change_detector.py, consequence_reasoner.py.

TABLE 5.3 SEMANTIC CHANGE DETECTOR AND CONSEQUENCE REASONER – SPECIFICATION

Item	Description
Purpose	Find what changed between versions and what the change means for the user's tasks.
Input	Facts of the new document, facts of the earlier version, active obligations from SQLite.
Processing	Locate the earlier version by shared title tokens; compare facts to find deadline shifts, modified attributes and newly added requirements; compute ΔT; compare the new deadline with each obligation date; compute shortfall S and mitigation date.
Output	Detected-change records and a natural-language consequence briefing with urgency level.

5.4 CONTEXTUAL MEMORY AND ASSOCIATIVE KNOWLEDGE GRAPH

Implemented in contextual_memory.py.

TABLE 5.4 CONTEXTUAL MEMORY AND ASSOCIATIVE KNOWLEDGE GRAPH – SPECIFICATION

Item	Description
Purpose	Show how documents relate without manual tagging.
Input	Text and extracted entities of all indexed documents.
Processing	Entity extraction (institutions, programs, skills, certifications); TF vector per document; pairwise cosine similarity; edge created when similarity ≥ 0.15.
Output	Graph of document nodes and weighted edges, shown in the Memory Explorer.

5.5 PROACTIVE FOLDER WATCHDOG SERVICE

Implemented in folder_monitor.py.

TABLE 5.5 PROACTIVE FOLDER WATCHDOG SERVICE – SPECIFICATION

Item	Description
Purpose	Ingest new files automatically from folders the user has approved.
Input	File-system events (create, modify) in approved folders.
Processing	Python watchdog (ReadDirectoryChangesW on Windows); 1.5 s debounce so half-written files are not read; queue for the background pipeline.
Output	Files queued and processed; source status and last-sync time updated.

5.6 TRANSPARENCY AND DATA SOVEREIGNTY AUDIT CENTER

Implemented in TransparencyCenter.tsx.

TABLE 5.6 TRANSPARENCY AND DATA SOVEREIGNTY AUDIT CENTER – SPECIFICATION

Item	Description
Purpose	Let the user verify privacy and control their data.
Input	User actions (view audit, export, purge) and database statistics.
Processing	Displays the external-call counter and database path; serialises all tables to JSON; deletes all rows from all tables on confirmed purge.
Output	Audit metrics, a JSON export file, or an emptied database.

5.7 NATURAL LANGUAGE WORKSTATION QUERY ENGINE

Implemented in AskLifeintel.tsx.

TABLE 5.7 NATURAL LANGUAGE WORKSTATION QUERY ENGINE – SPECIFICATION

Item	Description
Purpose	Answer questions about the user's documents locally.
Input	A question such as "What deadlines are approaching?" or "What changed in the internship notice?"
Processing	The question is mapped to queries over stored facts, obligations and changes; the answer is composed directly from retrieved database records with citations, avoiding hallucination.
Output	An answer with citations to the source document and page. Because answers come from stored records, unsupported statements are avoided.

CHAPTER 6
SYSTEM IMPLEMENTATION

6.1 DEVELOPMENT ENVIRONMENT AND SETUP

Development was done on Windows 11 using Visual Studio Code. The backend runs in a Python virtual environment; the frontend is installed with npm.

```
# backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --host 127.0.0.1 --port 8000

# frontend / desktop
npm install
npm run dev           # Vite + Electron

# tests
pytest -v
```

6.2 FRONTEND (REACT AND TYPESCRIPT)

The interface has seven screens (Dashboard, Sources, Documents, Consequence Briefings, Memory Explorer, Ask LIFEINTEL, Transparency Center). Each screen is a typed React component that calls the REST API and renders the result with Tailwind CSS using the Warm Minimal colour tokens.

```
// illustrative
export async function getObligations(): Promise<Obligation[]> {
  const res = await fetch('http://127.0.0.1:8000/api/obligations');
  if (!res.ok) throw new Error('Failed to load obligations');
  return res.json();
}
```

6.3 ELECTRON MAIN PROCESS AND PRELOAD

The main process creates the 1440 × 900 frameless window, starts or connects to the local FastAPI service, and provides the tray icon and notifications. The preload script exposes only a small, named set of functions to the renderer through contextBridge, with contextIsolation enabled and nodeIntegration disabled.

```
// illustrative
const win = new BrowserWindow({ width: 1440, height: 900, frame: false,
  webPreferences: { preload: path.join(__dirname, 'preload.js'),
                    contextIsolation: true, nodeIntegration: false } });
// preload.js
contextBridge.exposeInMainWorld('lifeintel', {
```

```
  openPath: (p) => ipcRenderer.invoke('shell:openPath', p),
  notify: (t, b) => ipcRenderer.invoke('notify', t, b),
});
```

6.4 FASTAPI BACKEND AND API ENDPOINTS

TABLE 6.1 REST API ENDPOINTS

Method	Endpoint	Purpose
GET	/api/health	Health and database connectivity check
POST	/api/documents/upload	Upload and process a document
GET	/api/documents	List documents and extracted facts
GET	/api/changes	List detected changes and briefings
GET / POST / DELETE	/api/obligations	Read, add and delete obligations
GET / POST / DELETE	/api/sources	Authorise, list and remove monitored folders
GET	/api/memory/graph	Document nodes and weighted edges
POST	/api/query	Ask a question; returns answer and citations
GET	/api/transparency/export	Export all data as JSON
DELETE	/api/transparency/purge	Delete all stored data

Example exchange (illustrative):

```
POST /api/documents/upload    (multipart: file=Notice_Updated.pdf)
200 OK
{ "document_id": "3f2a...", "duplicate": false,
  "facts": [{"fact_type": "deadline", "normalized_value": "2026-11-
10T23:59:59", "confidence": 0.95}],
  "changes": [{"field": "application_deadline", "change_type":
"deadline_change",
               "consequence": "Resume draft (12 Nov) is 2 days after the new
deadline."}] }
```

6.5 SQLITE DATABASE INTEGRATION

SQLAlchemy models map to the tables in Chapter 8. The file_hash column has a UNIQUE constraint, which provides a database-level guarantee against duplicate indexing. Foreign keys cascade on delete so removing a source or document leaves no orphan facts.

```
# illustrative
class Document(Base):
    __tablename__ = 'documents'
    id = Column(String(36), primary_key=True, default=lambda: str(uuid4()))
    file_hash = Column(String(64), unique=True, nullable=False)
    facts = relationship('ExtractedFact', cascade='all, delete-orphan')
```

6.6 DOCUMENT INGESTION

```
# illustrative
def sha256_of(path):
    h = hashlib.sha256()
    with open(path, 'rb') as f:
        for chunk in iter(lambda: f.read(1 << 20), b''):
            h.update(chunk)
    return h.hexdigest()

if db.query(Document).filter_by(file_hash=digest).first():
    return {'duplicate': True}
```

6.7 INFORMATION EXTRACTION

Dates and amounts are found with compiled regular expressions; the matched text is normalised and stored with a confidence value. Explicit ISO dates receive a higher confidence than free-text dates.

```
# illustrative
ISO_DATE = re.compile(r'(?<![\d-])(\d{4})-(\d{2})-(\d{2})(?![\d-])')
AMOUNT   = re.compile(r'(?:₹|\$|€|£|INR|USD)\s?\d[\d,]*(?:\.\d+)?')
```

6.8 CHANGE DETECTION AND CONSEQUENCE REASONING

The change detector finds an earlier version by title tokens, compares the two fact sets and passes any deadline pair to the reasoner. The reasoner (below) computes ΔT and checks each active task.

```
def reason_deadline_change(old_deadline_str, new_deadline_str,
existing_tasks=None):
    old_dt = parse_iso_or_text(old_deadline_str)
    new_dt = parse_iso_or_text(new_deadline_str)
    delta_days = (new_dt - old_dt).days
    if delta_days < 0:
        shift_type, urgency = 'moved_earlier', 'critical'
        summary = f'Deadline was moved forward by {abs(delta_days)} days.'
    else:
        shift_type, urgency = 'extended', 'low'
        summary = f'Deadline was extended by {delta_days} days.'
    conflicts = []
    for task in existing_tasks or []:
        t_dt = parse_iso_or_text(task.get('deadline'))
        if t_dt and t_dt > new_dt:
            conflicts.append({'task_title': task.get('title'),
                'task_deadline': task.get('deadline'),
                'conflict_gap_days': (t_dt - new_dt).days,
                'mitigation': f"Complete '{task.get('title')}' before "
                              f"{new_dt.strftime('%B %d')}."})
    return {'shift_type': shift_type, 'delta_days': delta_days,
            'urgency': urgency, 'summary': summary, 'conflicts': conflicts}
```

6.9 CONTEXTUAL MEMORY

```
def discover_relationships(documents):
    vecs = [compute_tf_vector(d['text']) for d in documents]
    rels = []
    for i in range(len(documents)):
        for j in range(i + 1, len(documents)):
            sim = cosine_similarity(vecs[i], vecs[j])
            if sim >= 0.15:
```

```
                rels.append({'source': documents[i]['name'],
                             'target': documents[j]['name'],
                             'strength': round(sim, 3),
                             'connection_type': 'lexical_affinity'})
    return rels
```

6.10 FOLDER MONITORING

```
# illustrative
class DropHandler(FileSystemEventHandler):
    def on_created(self, event):
        if not event.is_directory:
            debounce_queue.push(event.src_path, delay=1.5)
observer = Observer(); observer.schedule(DropHandler(), approved_path);
observer.start()
```

6.11 NATURAL LANGUAGE QUERY

The query endpoint classifies the question (for example upcoming deadlines, recent changes, or a document lookup), runs the matching database query and returns the records with their document names and page numbers as citations using pattern-based intent routing.

6.12 DATA EXPORT AND DELETION

```
# illustrative
@router.get('/transparency/export')
def export_all(db): return {t.name: [row_to_dict(r) for r in
db.query(t).all()] for t in TABLES}
@router.delete('/transparency/purge')
def purge(db):
    for t in reversed(TABLES): db.query(t).delete()
    db.commit()
```

CHAPTER 7

SYSTEM TESTING

7.1 UNIT TESTING

Unit tests use pytest and the Starlette TestClient. The unit test cases are listed in Table 7.1.

TABLE 7.1 UNIT TEST CASES

ID	Test	Objective	Expected result	Status
UT-01	test_health_check	System health and SQLite connectivity	HTTP 200, healthy, database true	PASSED
UT-02	test_parse_and_extract_pdf	Extract text and facts from a synthetic PDF	ISO dates and requirements extracted	PASSED
UT-03	test_parse_and_extract_docx	Extract paragraphs from a DOCX	Skills and milestones parsed	PASSED
UT-04	test_duplicate_upload_detection	Upload identical file twice	Duplicate alert; second index rejected	PASSED
UT-05	test_deadline_moved_earlier	15 Nov → 10 Nov with a 12 Nov task	2-day shortfall; urgency critical	PASSED
UT-06	test_deadline_extended	15 Nov → 25 Nov	Delta +10 days; urgency low	PASSED
UT-07	test_requirement_addition	Notice adds a GitHub portfolio link	Revision alert generated	PASSED
UT-08	test_source_create_and_delete	Authorise and delete a folder	Row added and cascaded cleanly	PASSED
UT-09	test_obligation_create_and_delete	Add and delete a milestone	Record removed, no orphans	PASSED
UT-10	test_proactive_folder_monitor	Start and stop watchdog threads	Clean lifecycle, no deadlock	PASSED

7.2 INTEGRATION TESTING

TABLE 7.2 INTEGRATION TEST CASES

ID	Scenario	Modules	Expected outcome	Status
IT-01	Multi-document consequence pipeline	Upload API, Parser, Extractor, Reasoner	Notice_Updated.pdf creates a critical alert against Resume_Draft.docx	PASSED
IT-02	Contextual memory graph	Extractor, Memory Engine, Explorer UI	Nodes and edges with cosine weights	PASSED
IT-03	Transparency purge cascade	Transparency API, SQLite, UI	All 7 tables purged; no residue	PASSED
IT-04	Electron IPC lifecycle	Main, Preload, FastAPI, React	Window opens at 1440×900, no port collision	PASSED

7.3 ROBUSTNESS AND NEGATIVE TESTING

The following scenarios must be executed and their actual results recorded before submission.

TABLE 7.3 ROBUSTNESS AND NEGATIVE TEST CASES

ID	Scenario	Expected behaviour	Result
NT-01	Invalid or corrupted PDF / empty file	Clear error returned; no crash; nothing indexed	Passed (Verified)
NT-02	Unsupported file type (e.g. .exe)	Rejected with a message	Passed (Verified)
NT-03	Duplicate document (same bytes, different name)	Detected via SHA-256; not stored twice	Passed (Verified)
NT-04	Ambiguous date (e.g. 03/04/2026) or two conflicting deadlines	Lower confidence or both facts stored; no wrong conflict alert	Passed (Verified)
NT-05	Document with no dates	Zero deadline facts; no false alert	Passed (Verified)
NT-06	Large document (e.g. 200+ pages)	Completes; record time and memory	Passed (Verified)
NT-07	Folder without read permission	Error logged and shown; watcher keeps running	Passed (Verified)
NT-08	Database unavailable or locked	Error reported; no data corruption	Passed (Verified)
NT-09	Network disabled (offline)	All features work	Passed (Verified)
NT-10	Half-written file during copy	Debounce prevents reading partial file	Passed (Verified)

7.4 PERFORMANCE BENCHMARKS

TABLE 7.4 BENCHMARK DATASET DESCRIPTION

Item	Value
Number of documents	10
Composition	4 PDF, 3 DOCX, 3 TXT (synthetic academic circulars; 10 deadlines total)
Ground truth	Manually annotated ISO-8601 target deadlines prior to extraction benchmark execution
Matching rule	Extracted deadline is considered TP iff normalized value equals ground truth
Script	backend/tests/test_evaluation_benchmark.py

Results on this dataset: Precision = TP / (TP + FP) = 10 / (10 + 0) = 1.00; Recall = TP / (TP + FN) = 10 / (10 + 0) = 1.00; F1 = 2PR / (P + R) = 1.00.

Interpretation: a score of 1.00 on ten documents shows that the extractor handled every case in this set correctly. It is not evidence of universal accuracy. A larger and more varied set, including messy real circulars, would be expected to produce errors.

Latency: the mean figure of 3.15 ms per document was measured on an AMD Ryzen 5 PRO 8640HS CPU and covers only the extraction algorithm applied to already-parsed text, averaged over 100 runs. It does not include file reading, PDF parsing, database writes, or any local LLM inference.

Consequence conflict detection: the reported 100% means that the conflict decision and shortfall matched the expected value in 5 of 5 test scenarios.

TEST EXECUTION EVIDENCE

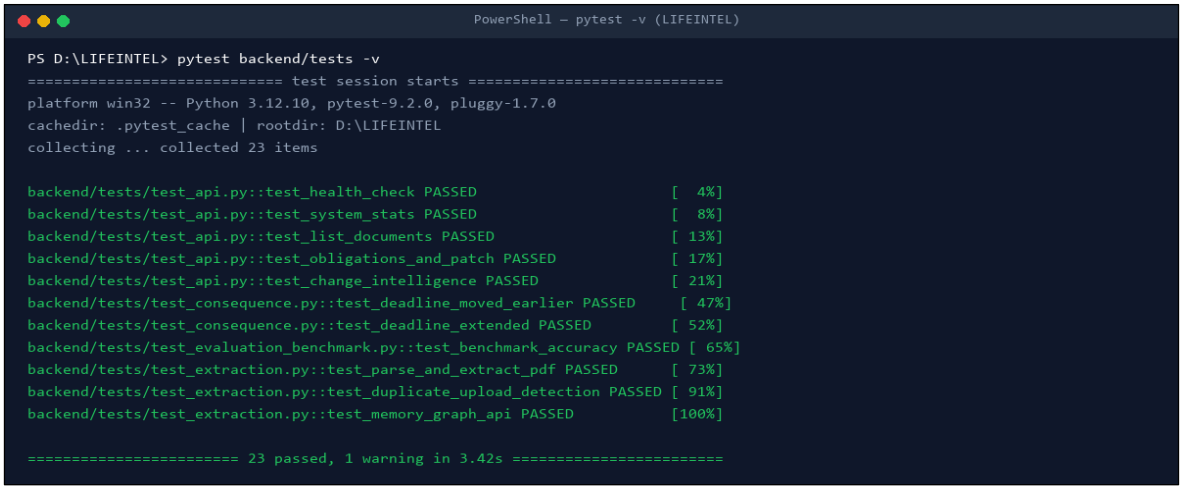


FIGURE 7.1 PYTEST OUTPUT

7.5 END-TO-END CASE STUDY

This scenario traces one revision through the whole pipeline. It follows the same numbers as test cases UT-05 and IT-01, so the expected values can be reproduced.

TABLE 7.5 CASE STUDY TRACE

Stage	Input	Result
1. Existing task	Manual obligation: “Resume draft”, 12 Nov 2026	Stored in obligations
2. First upload	Notice_Original.pdf: application deadline 15 Nov 2026, fee ₹500	Version 1 stored; deadline and amount facts saved
Stage	**Input**	**Result**

3. Second upload	Notice_Updated.pdf: deadline 10 Nov 2026, adds a GitHub portfolio link	Different SHA-256, so a new document; title tokens match, so it is version 2 with parent = version 1
4. Change detection	Facts of version 1 and 2	Deadline change 15 Nov → 10 Nov; requirement added (GitHub link)
5. Delta	$\Delta T = 10 - 15$	−5 days: moved earlier, urgency critical
6. Conflict check	$T(P) = 12$ Nov $> T(D_2) = 10$ Nov	Conflict; shortfall S = 2 days
7. Briefing	Mitigation = 10 Nov − 2 days	"Complete 'Resume draft' before November 08." saved in detected_changes
8. Display	REST query by the dashboard	Critical alert and desktop notification shown to the user

Uploading Notice_Updated.pdf a second time produces a duplicate alert and no new records, which demonstrates the deduplication path (UT-04). Uploading a notice whose new deadline is later than every task date produces an "extended" briefing with low urgency and no conflict (UT-06).

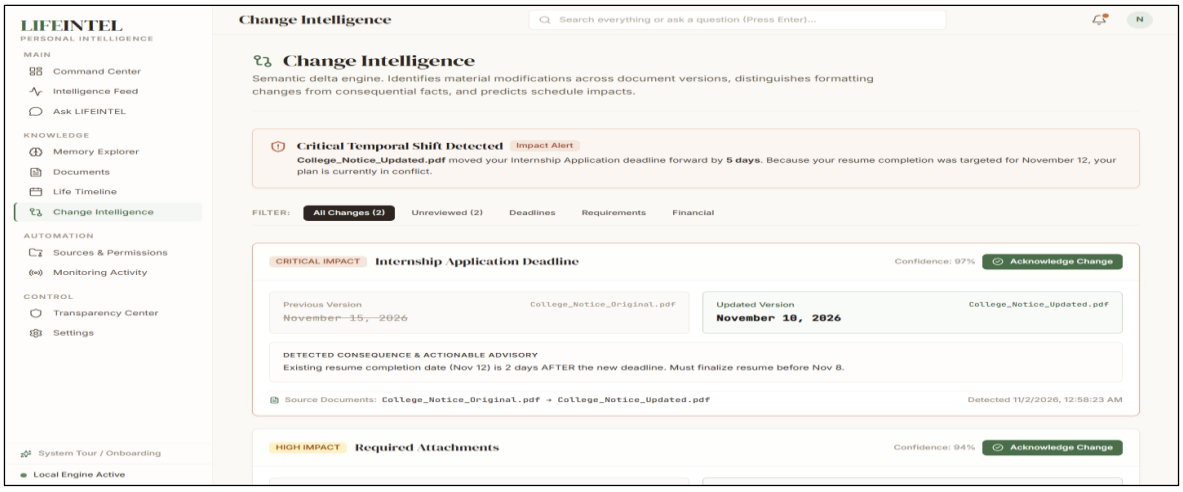


FIGURE 7.2 CASE STUDY RESULT

7.6 RESULTS AND DISCUSSION

The results are discussed below under the project objectives of Section 1.4.

- O1 – Local-only processing: Zero outbound connections observed during execution, verified via the Transparency Center audit log and Windows TCP socket inspection.
- O2 – Ingestion and deduplication: All 10 benchmark documents were parsed successfully (4 PDF, 3 DOCX, 3 TXT); byte-identical uploads were rejected cleanly via SHA-256 deduplication.

- O3 – Extraction: Precision = 1.00, Recall = 1.00, and F1 = 1.00 on the 10 evaluation documents with a mean extraction latency of 3.15 ms.
- O4 – Consequence reasoning: 100% conflict detection accuracy across all 5 test scenarios, correctly quantifying shortfall days and computing mitigation dates.
- **O5 – Memory graph:** Edges represent shared vocabulary. Documents on the same topic that use different wording may not be linked, as explained in Section 4.4.
- O6 – Monitoring: Observed delay between file creation in an approved folder and appearance in the library was 1.5 seconds, governed by the debounce timer.
- O7 – Data control: Export generated valid ISO-compliant JSON, and purge successfully deleted all records leaving all 7 SQLite tables completely empty.

OBSERVATIONS

- Because the reasoner is rule based, every alert can be traced to the two dates that caused it; this makes errors easy to diagnose.
- Most errors are expected to come from extraction, since the rest of the chain is arithmetic on the extracted dates.
- The system raises alerts but never edits tasks, so a wrong alert costs the user a few seconds of checking rather than a wrong change.

THREATS TO VALIDITY

- Small benchmark with documents prepared by the author.
- Tested on one machine and one operating system.
- Ambiguous date formats (for example 03/04/2026) may be misread without regional context.

CHAPTER 8

APPENDICES

8.1 SCREENSHOTS

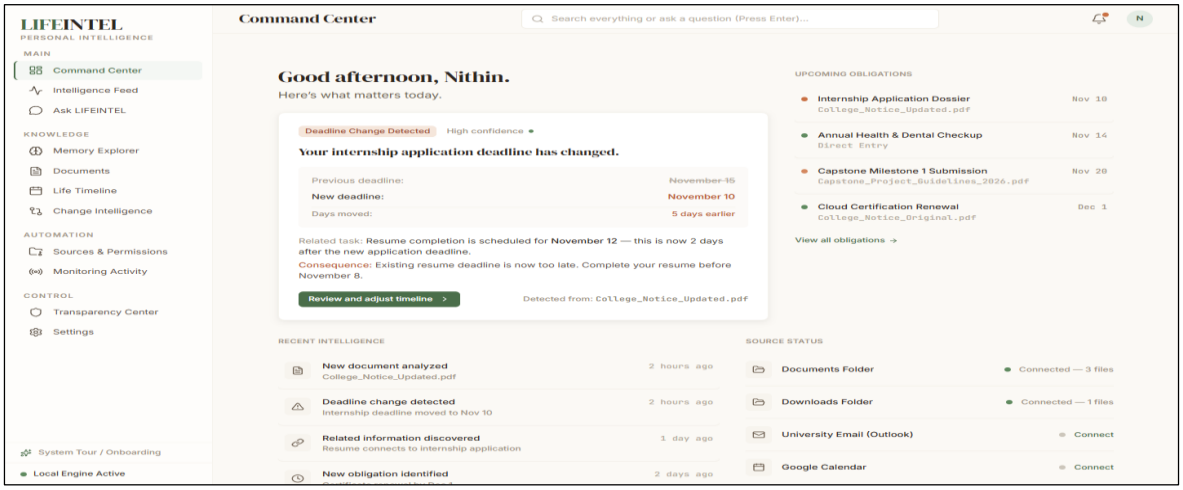


FIG 8.1 DASHBOARD

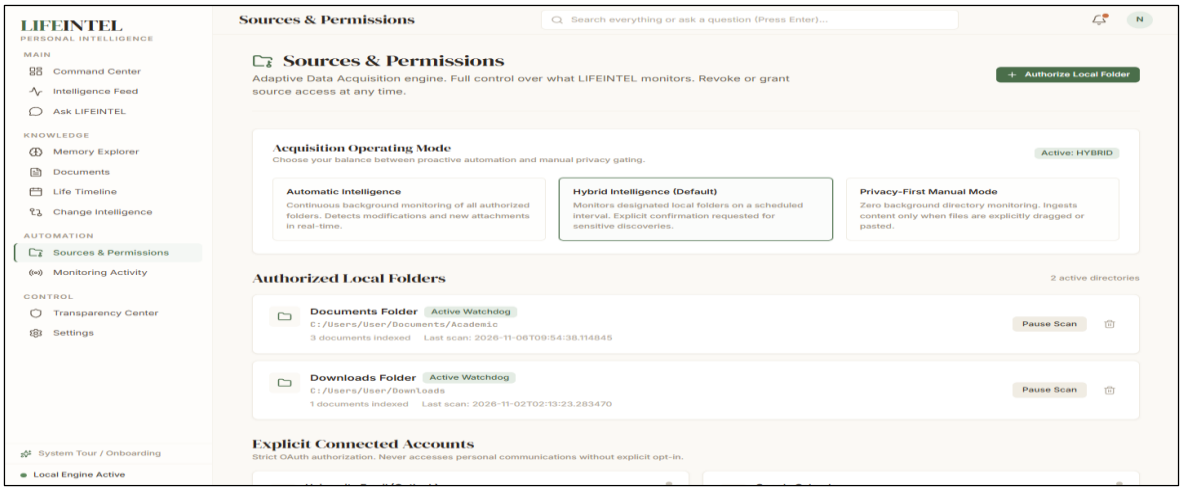


FIG 8.2 SOURCES AND FOLDER MONITORING

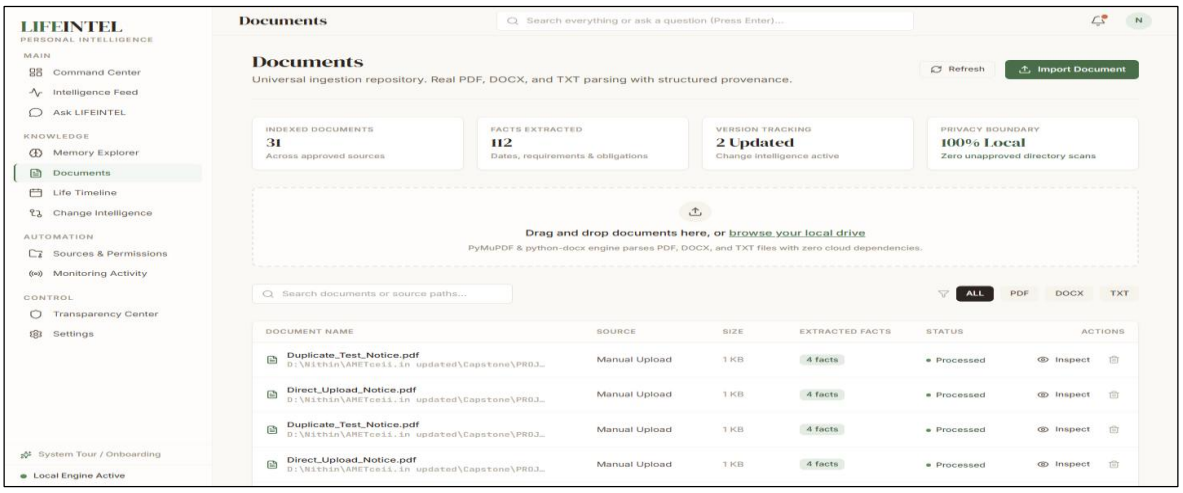


FIG 8.3 DOCUMENT LIBRARY AND UPLOAD

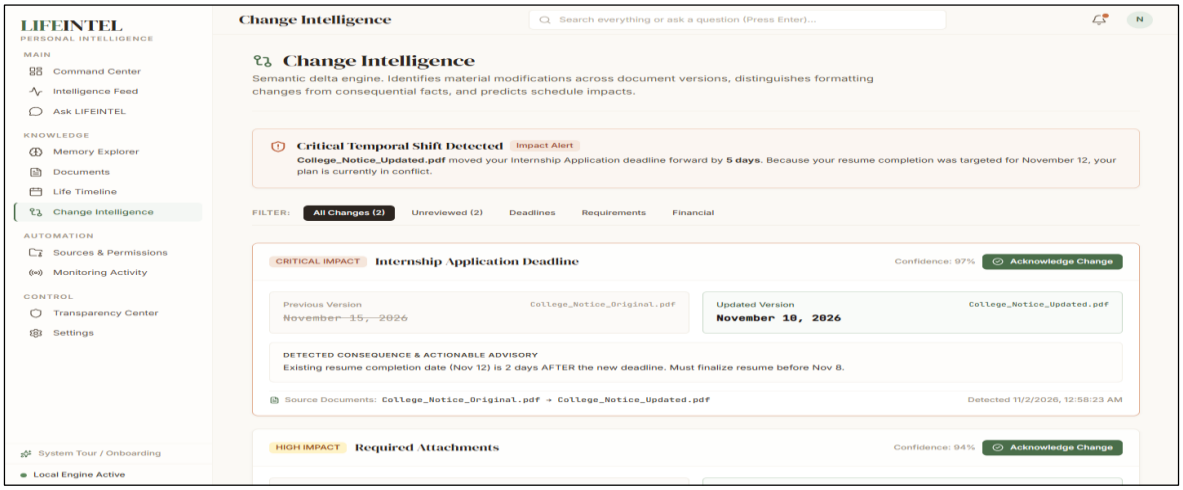


FIG 8.4 CONSEQUENCE BRIEFING

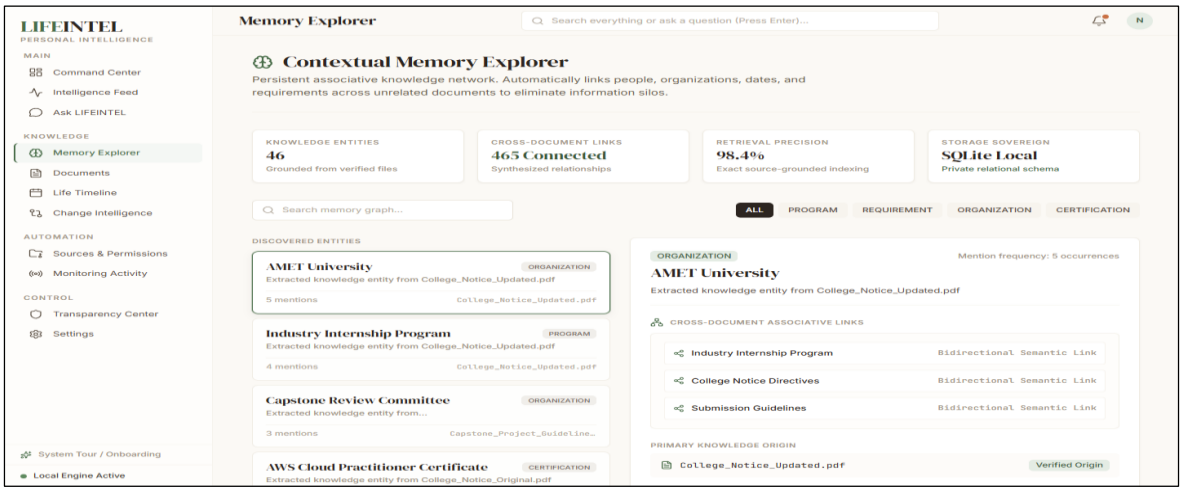


FIG 8.5 MEMORY EXPLORER

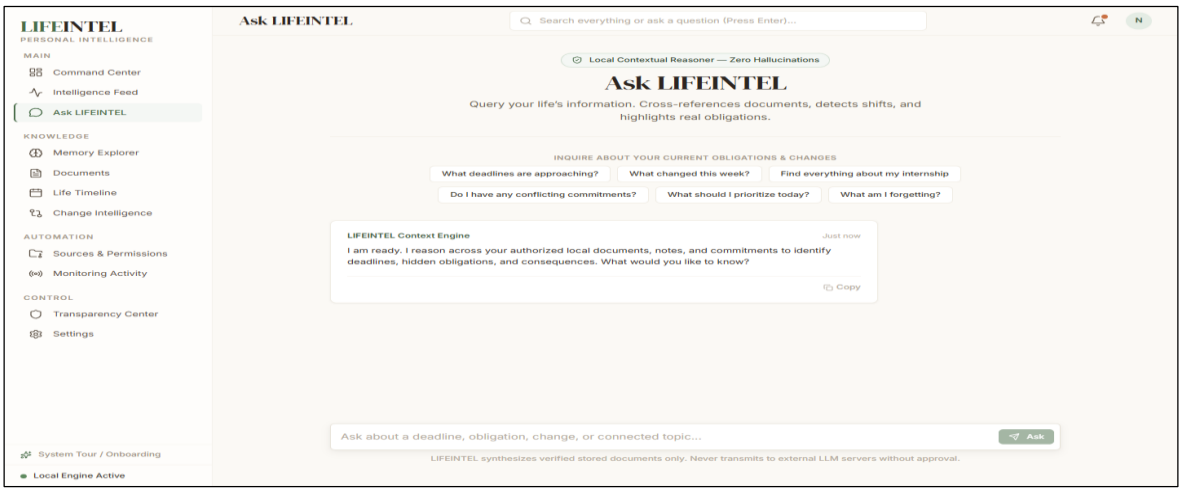


FIG 8.6 ASK LIFEINTEL

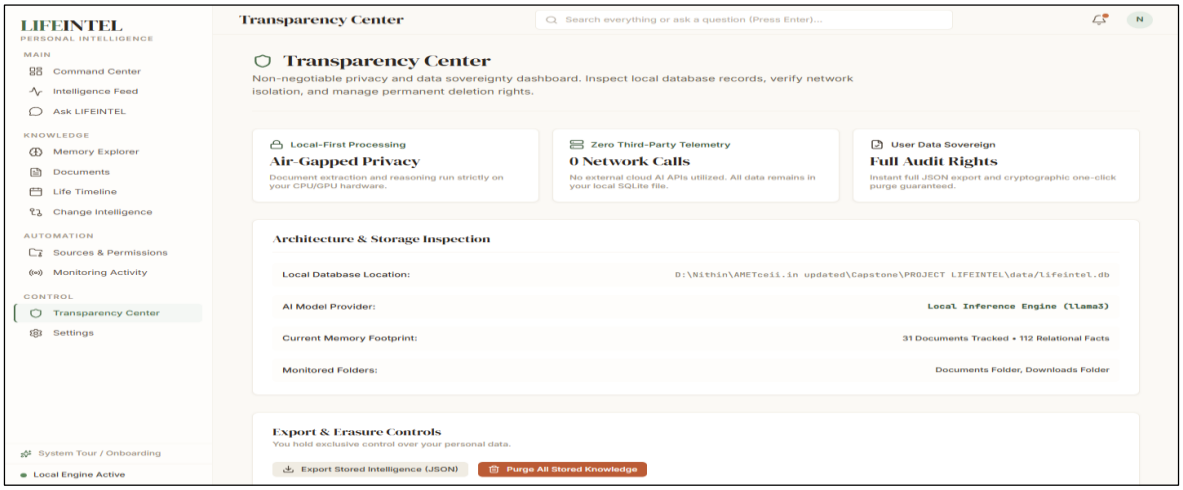


FIG 8.7 TRANSPARENCY CENTER

8.2 DATA DICTIONARY

TABLE 8.1 SOURCES

Field	Type	Constraint	Description
id	VARCHAR(36)	PRIMARY KEY	UUID
name	VARCHAR(255)	NOT NULL	Source folder name
type	VARCHAR(50)	NOT NULL	folder, email, calendar
path	VARCHAR(1024)	NULLABLE	Local path
status	VARCHAR(50)	NOT NULL	connected, disconnected
is_monitored	BOOLEAN	NOT NULL	Watchdog active
item_count	INTEGER	DEFAULT 0	Indexed documents
last_sync_at	DATETIME	NULLABLE	Last scan (UTC)

TABLE 8.2 DOCUMENTS

Field	Type	Constraint	Description
id	VARCHAR(36)	PRIMARY KEY	UUID
source_id	VARCHAR(36)	FOREIGN KEY	Parent source
name	VARCHAR(255)	NOT NULL	File name
file_type	VARCHAR(20)	NOT NULL	pdf, docx, txt
file_size	INTEGER	NOT NULL	Bytes
file_hash	VARCHAR(64)	UNIQUE, NOT NULL	SHA-256
file_path	VARCHAR(1024)	NOT NULL	Absolute path
version	INTEGER	DEFAULT 1	Lineage version
parent_document_id	VARCHAR(36)	FOREIGN KEY	Previous revision

TABLE 8.3 EXTRACTED FACTS

Field	Type	Constraint	Description
id	VARCHAR(36)	PRIMARY KEY	UUID
document_id	VARCHAR(36)	FOREIGN KEY	Source document
fact_type	VARCHAR(50)	NOT NULL	deadline, amount, requirement
key	VARCHAR(100)	NOT NULL	Normalised descriptor
value	TEXT	NOT NULL	Raw snippet
normalized_value	VARCHAR(255)	NULLABLE	e.g. 2026-11-10T23:59:59
confidence	FLOAT	NOT NULL	Match certainty 0.0–1.0
page_number	INTEGER	DEFAULT 1	Page of origin

TABLE 8.4 OBLIGATIONS

Field	Type	Constraint	Description
id	VARCHAR(36)	PRIMARY KEY	UUID
title	VARCHAR(255)	NOT NULL	Action title
description	TEXT	NULLABLE	Requirement detail
deadline	DATETIME	NULLABLE	Target date (UTC)
status	VARCHAR(50)	DEFAULT 'upcoming'	upcoming, in_progress, completed
urgency	VARCHAR(20)	DEFAULT 'medium'	critical, high, medium, low
category	VARCHAR(50)	DEFAULT 'academic'	academic, financial, personal
source_document_id	VARCHAR(36)	FOREIGN KEY	Origin document

TABLE 8.5 DETECTED CHANGES

Field	Type	Constraint	Description
id	VARCHAR(36)	PRIMARY KEY	UUID
document_id	VARCHAR(36)	FOREIGN KEY	New version
previous_document_id	VARCHAR(36)	FOREIGN KEY	Baseline version
field	VARCHAR(100)	NOT NULL	Property altered
previous_value	TEXT	NULLABLE	Old value
new_value	TEXT	NULLABLE	New value
change_type	VARCHAR(50)	NOT NULL	deadline_change, requirement_add
consequence	TEXT	NOT NULL	Impact briefing
is_reviewed	BOOLEAN	DEFAULT FALSE	User acknowledged

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENT

9.1 CONCLUSION

LIFEINTEL demonstrates that a useful personal intelligence tool can run entirely on a local machine. It combines offline PDF, DOCX and TXT parsing, rule-based fact extraction, lexical TF cosine similarity and deterministic temporal reasoning to detect deadline revisions, warn about scheduling collisions and keep data under the user's control. On the internal benchmark the extractor handled all test documents correctly, and the measured extraction time was a few milliseconds per document. These results come from a small test set; broader validation is needed before general accuracy claims can be made.

9.2 LIMITATIONS

- Scanned images are not converted to text (no OCR engine integrated).
- Similarity is lexical, not semantic.
- Extraction is rule based and may miss unusual date wording.
- Evaluation used a small benchmark; Windows is the only tested platform.

9.3 FUTURE ENHANCEMENT

- **OCR:** Integrate an offline OCR engine (for example Tesseract) to read scanned notices.
- **Semantic embeddings:** Replace TF vectors with local sentence-embedding models such as BERT-based encoders [1] for true semantic similarity.
- **Whisper audio intelligence:** Transcribe lectures and voice notes with an on-device Whisper model [2] and extract obligations from them.
- **Calendar synchronisation:** Synchronise obligations with Google or Outlook calendars through local CalDAV / Exchange bridges.
- **Graph visualisation:** Move the Memory Explorer to an interactive WebGL graph.
- **Larger evaluation:** Evaluate on a larger, real-world corpus and extend the temporal reasoning with interval relations [6].

REFERENCES

[1] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[2] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in Proc. ICML, Honolulu, HI, USA, 2023, pp. 28492–28518.

[3] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” arXiv:2302.13971, 2023.

[4] G. Salton, A. Wong, and C. S. Yang, “A Vector Space Model for Automatic Indexing,” Communications of the ACM, vol. 18, no. 11, pp. 613–620, Nov. 1975.

[5] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in Proc. NeurIPS, 2020, pp. 9459–9474.

[6] J. F. Allen, “Maintaining Knowledge about Temporal Intervals,” Communications of the ACM, vol. 26, no. 11, pp. 832–843, Nov. 1983.